

# Static Malware Analyzer

---

## Comprehensive Technical Documentation

A Static Malware Analysis System  
Built with FastAPI, MongoDB, and MinIO

Version 1.0  
February 2026

## Table of Contents

1. 1. Executive Summary
2. 2. System Architecture
3. 2.1 High-Level Architecture
4. 2.2 Technology Stack
5. 2.3 Data Flow
6. 3. Project Structure
7. 3.1 Directory Organization
8. 3.2 Package Descriptions
9. 4. Core Components
10. 4.1 API Layer (main.py)
11. 4.2 Storage Layer (minio.py)
12. 4.3 Database Models
13. 5. Services Layer
14. 5.1 Hash Service
15. 5.2 IOC Extractor
16. 5.3 YARA Analyzer
17. 5.4 Report Generator
18. 6. Schemas and Models
19. 6.1 Enumerations
20. 6.2 Analysis Schemas
21. 6.3 YARA Schemas
22. 6.4 IOC Schemas
23. 7. Detection Rules
24. 7.1 YARA Rules Overview
25. 7.2 Rule Examples
26. 8. Key Features
27. 9. Dependencies and Libraries
28. 10. API Documentation
29. 11. Security Considerations
30. 12. Conclusion

## 1. Executive Summary

StaticMalwareAnalyzer is a sophisticated malware analysis system designed to perform comprehensive static analysis of Windows Portable Executable (PE) files. The system leverages modern microservices architecture, combining FastAPI for the REST API layer, MongoDB for metadata storage, and MinIO for distributed object storage.

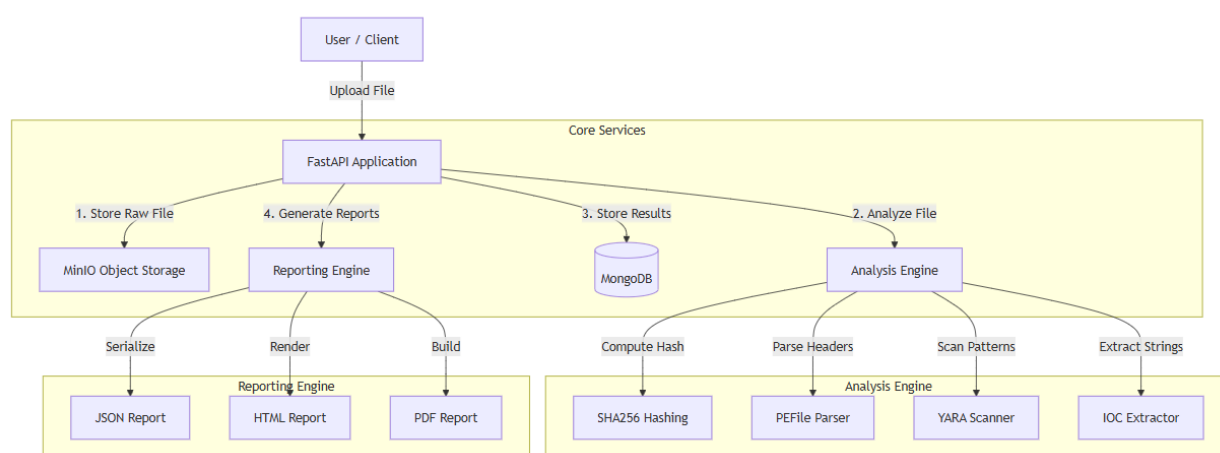
### Key Capabilities

- **PE File Validation:** Validates Windows executable file format and structure
- **Hash-Based Detection:** Computes SHA-256 hashes and checks against known malware databases
- **Structural Analysis:** Analyzes PE sections, imports, and compilation timestamps
- **Entropy Analysis:** Detects packed or encrypted sections using entropy calculations
- **YARA Rule Matching:** Signature-based detection using extensible YARA rules
- **IOC Extraction:** Automatic extraction of Indicators of Compromise (URLs, IPs, domains, registry keys)
- **Multi-Format Reporting:** Generates JSON, HTML, and PDF reports with risk scoring
- **MITRE ATT&CK Mapping:** Maps detected threats to MITRE ATT&CK framework techniques

## 2. System Architecture

### 2.1 High-Level Architecture

The system follows a microservices-like architecture with clear separation of concerns. The architecture diagram below illustrates the complete workflow from file upload to report generation:



The workflow consists of four main phases:

31. 1. Store Raw File: Files are uploaded and stored in MinIO object storage
32. 2. Analyze File: The Analysis Engine performs comprehensive PE analysis
33. 3. Store Results: Analysis results and metadata are stored in MongoDB
34. 4. Generate Reports: Multi-format reports are created and stored

## 2.2 Technology Stack

| Category            | Technology  | Purpose                                  |
|---------------------|-------------|------------------------------------------|
| Web Framework       | FastAPI     | Asynchronous REST API server             |
| Database            | MongoDB     | Metadata and analysis results storage    |
| Object Storage      | MinIO       | Distributed file storage (S3-compatible) |
| ODM                 | Beanie      | MongoDB Object Document Mapper           |
| PE Analysis         | pefile      | Parsing and analyzing PE file structures |
| Signature Detection | yara-python | Pattern-based malware detection          |
| Templating          | Jinja2      | HTML report generation                   |
| PDF Generation      | ReportLab   | PDF document creation                    |
| Async Runtime       | asyncio     | Asynchronous operation handling          |
| Validation          | Pydantic    | Data validation and serialization        |

## 2.3 Data Flow

The data flow through the system follows a well-defined pipeline:

35. **Client Upload:** User uploads a suspicious file via POST request to the API endpoint
36. **File Storage:** File is temporarily saved and uploaded to MinIO object storage
37. **Hash Calculation:** SHA-256 hash is computed for the file
38. **Blacklist Check:** Hash is checked against known malware database (bad\_hashes.txt)
39. **PE Validation:** File is validated as a Windows PE executable
40. **Structural Analysis:** PE headers, sections, and imports are analyzed
41. **Entropy Calculation:** Section entropy is calculated to detect packing/encryption
42. **String Extraction:** ASCII strings are extracted from the binary
43. **YARA Scanning:** File is scanned against all loaded YARA rules
44. **IOC Extraction:** Regular expressions extract IOCs from strings
45. **Report Generation:** JSON, HTML, and PDF reports are generated (optional)
46. **Database Storage:** Results are stored in MongoDB with file metadata
47. **Response:** Complete analysis results are returned to the client

## 3. Project Structure

### 3.1 Directory Organization

The project follows a modular structure with clear separation of concerns:

```
StaticMalwareAnalyzer/
├── app/
│   ├── const/
│   │   └── constant.py          # Application constants
│   ├── data/
│   │   └── bad_hashes.txt      # Known malware hash database
│   ├── models/
│   │   └── file.py            # MongoDB document models
│   ├── rules/
│   │   ├── C2_Network_Beaconing.yar
│   │   ├── CoinMiner_Generic.yar
│   │   └── [other YARA rules]
│   ├── schema/
│   │   ├── analyser.py        # Base analysis schemas
│   │   ├── enum.py            # Enumerations
│   │   ├── hashing.py         # PE analysis schemas
│   │   ├── IOC.py             # IOC extraction schemas
│   │   └── yara.py            # YARA result schemas
│   ├── services/
│   │   ├── hash.py            # Core PE analysis service
│   │   ├── ioc.py             # IOC extractor service
│   │   ├── report_generator.py # Report generation service
│   │   └── yara_analyzer.py    # YARA scanning service
│   └── templates/
│       └── report_template.html # HTML report template
├── reports/                    # Generated reports output
├── .venv/                      # Virtual environment
├── main.py                     # FastAPI application entry point
├── minio.py                    # MinIO storage wrapper
├── create_pe.py                # PE file generator (testing)
├── docker-compose.yml          # Docker services configuration
├── Dockerfile                  # Application container
├── pyproject.toml              # Python project configuration
├── README.md                   # Project documentation
└── LICENSE                     # MIT License
```

## 3.2 Package Descriptions

| Package               | Description                                                                             |
|-----------------------|-----------------------------------------------------------------------------------------|
| <b>app/const/</b>     | Application-wide constants including file paths, buffer sizes, and suspicious API lists |
| <b>app/data/</b>      | Static data files including the malware hash blacklist database                         |
| <b>app/models/</b>    | Beanie ODM models for MongoDB document structure                                        |
| <b>app/rules/</b>     | YARA rule files for signature-based malware detection                                   |
| <b>app/schema/</b>    | Pydantic schemas for data validation and serialization                                  |
| <b>app/services/</b>  | Business logic layer containing core analysis services                                  |
| <b>app/templates/</b> | Jinja2 templates for HTML report generation                                             |



## 4. Core Components

### 4.1 API Layer (main.py)

The main.py file serves as the application entry point and defines the FastAPI application with all route handlers and lifecycle management.

#### Key Functions:

| Function                    | Parameters                                 | Description                                                                                  |
|-----------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>init_db()</b>            | None                                       | Initializes MongoDB connection and Beanie ODM with FileModel document                        |
| <b>init_minio()</b>         | None                                       | Initializes MinIO client for object storage operations                                       |
| <b>lifespan()</b>           | app: FastAPI                               | Manages application startup and shutdown lifecycle events                                    |
| <b>upload_and_analyze()</b> | file: UploadFile,<br>generate_report: bool | Main endpoint: uploads file, performs analysis, stores results, optionally generates reports |

#### Workflow:

- Receive file upload via multipart form data
- Save file temporarily and upload to MinIO
- Compute SHA-256 hash and check blacklist
- Execute comprehensive PE analysis
- Generate reports if requested
- Upload reports to MinIO and get presigned URLs
- Create FileModel document with all results
- Insert document into MongoDB
- Return complete analysis to client

## 4.2 Storage Layer (minio.py)

The FolderBucket class provides a clean abstraction over MinIO operations, managing object storage with automatic bucket creation and UUID-based file naming.

| Method                 | Parameters                                               | Description                                         |
|------------------------|----------------------------------------------------------|-----------------------------------------------------|
| <b>list_buckets()</b>  | None                                                     | Returns list of all available buckets               |
| <b>list_objects()</b>  | bucket_name: str,<br>prefix: str                         | Lists objects in bucket with optional prefix filter |
| <b>ensure_bucket()</b> | bucket_name: str                                         | Creates bucket if it does not exist                 |
| <b>upload_file()</b>   | bucket_name: str,<br>file_path: str                      | Uploads file and returns UUID object name           |
| <b>download_file()</b> | bucket_name: str,<br>object_name: str,<br>file_path: str | Downloads object to local path                      |

### Storage Architecture:

- Files Bucket: Stores uploaded malware samples with UUID filenames
- Reports Bucket: Stores generated JSON, HTML, and PDF reports
- Presigned URLs: Temporary access URLs for downloading reports
- Object Metadata: Additional context stored with each object

### 4.3 Database Models

The File model (app/models/file.py) represents the MongoDB document structure for storing file metadata and analysis results.

| Field                | Type           | Description                     |
|----------------------|----------------|---------------------------------|
| <b>file_id</b>       | str            | MinIO object ID or storage path |
| <b>filename</b>      | str            | Original filename from upload   |
| <b>size</b>          | str            | File size in bytes              |
| <b>hash</b>          | str (indexed)  | SHA-256 hash for quick lookups  |
| <b>content_type</b>  | Optional[str]  | MIME type of the file           |
| <b>headers</b>       | Optional[Dict] | HTTP headers from upload        |
| <b>uploaded_at</b>   | datetime       | Timestamp of upload (UTC)       |
| <b>analysis</b>      | Optional[Dict] | Complete FileAnalysis results   |
| <b>Yara _results</b> | Optional[Dict] | YARA matching results           |

## 5. Services Layer

### 5.1 Hash Service (app/services/hash.py)

The hash service is the core analysis engine, orchestrating all PE analysis operations. It provides comprehensive static analysis capabilities.

#### Primary Functions:

| Function                           | Description                                                              |
|------------------------------------|--------------------------------------------------------------------------|
| <b>compute_hash(file_path)</b>     | Calculates SHA-256 hash using 64KB chunked reading for memory efficiency |
| <b>is_problematic(file_hash)</b>   | Checks hash against bad_hashes.txt blacklist database                    |
| <b>is_pe(path)</b>                 | Validates PE format by checking MZ header and PE signature               |
| <b>validate_file_size(path)</b>    | Ensures file size is within 1KB to 50MB range                            |
| <b>get_compile_time(pe)</b>        | Extracts compilation timestamp from PE header                            |
| <b>is_timestamp_suspicious(dt)</b> | Detects anomalous timestamps (pre-1995 or future dates)                  |
| <b>analyze_sections(pe)</b>        | Analyzes all PE sections, calculating entropy for each                   |
| <b>analyze_imports(pe)</b>         | Extracts imported DLLs and functions, flags suspicious APIs              |
| <b>has_signature(pe)</b>           | Checks for digital signature in Security Directory                       |
| <b>extract_strings(path)</b>       | Extracts printable ASCII strings using "strings" command                 |
| <b>analyze_file(path)</b>          | Main orchestrator: executes all analysis steps and returns FileAnalysis  |

#### Suspicious API Detection:

The service maintains a list of Windows APIs commonly used by malware:

- Memory Operations: VirtualAlloc, VirtualProtect, WriteProcessMemory
- Process Injection: CreateRemoteThread

- Registry Manipulation: RegSetValueEx, RegCreateKeyEx
- Network Operations: InternetConnect, URLDownloadToFile, socket, connect
- Process Execution: CreateProcess, WinExec, ShellExecute
- Privilege Escalation: OpenProcessToken, AdjustTokenPrivileges

### Entropy Analysis:

Entropy is calculated for each PE section to detect packing or encryption. Sections with entropy  $\geq 7.5$  are flagged as suspicious, indicating potential compression or encryption commonly used by malware to evade detection.

## 5.2 IOC Extractor (app/services/ioc.py)

The IOCExtractor class uses regular expressions to identify Indicators of Compromise within extracted strings from the binary.

| IOC Type        | Pattern Description                                                 |
|-----------------|---------------------------------------------------------------------|
| URLs            | Matches http:// and https:// URLs with complete path components     |
| IP Addresses    | Matches IPv4 addresses in dotted-decimal notation                   |
| Domain Names    | Matches domains with common TLDs (.com, .net, .org, .ru, .cn, etc.) |
| Email Addresses | Matches standard email format with @ symbol                         |
| Registry Keys   | Matches Windows registry paths (HKEY_*)                             |
| File Paths      | Matches Windows file paths (C:\path\to\file.ext)                    |

### Method:

**Extract (text: str) -> IOCResult**

Takes extracted strings as input, applies all regex patterns, deduplicates results, and returns an IOCResult object containing all discovered indicators.

## 5.3 YARA Analyzer (app/services/yara\_analyzer.py)

The YaraAnalyzer class manages YARA rule compilation and file scanning for signature-based detection.

| Method                            | Description                                                             |
|-----------------------------------|-------------------------------------------------------------------------|
| <code>__init__()</code>           | Initializes analyzer and loads all YARA rules from app/rules/ directory |
| <code>_load_rules()</code>        | Discovers .yar and .yara files, compiles them into a single ruleset     |
| <code>scan_file(file_path)</code> | Scans file against all rules, returns detailed match information        |

### Match Information:

For each matched rule, the analyzer returns:

- Rule Name: Identifier of the matched YARA rule
- Tags: Any tags associated with the rule
- Metadata: Author, description, family, severity from rule definition
- String Matches: Specific strings that triggered the match, including byte offsets

### Rule Loading:

Rules are loaded at initialization from the app/rules/ directory. The system automatically compiles all .yar and .yara files into a single optimized ruleset. If no rules are found or compilation fails, a RuntimeError is raised.

## 5.4 Report Generator (app/services/report\_generator.py)

The report generator creates professional, multi-format reports from analysis results, including risk scoring and MITRE ATT&CK framework mapping.

### Main Function:

**generate\_reports(analysis, output\_dir, file\_path)**

Generates JSON, HTML, and PDF reports with comprehensive analysis results.

### Report Formats:

| Format | Description                                                    |
|--------|----------------------------------------------------------------|
| JSON   | Complete machine-readable analysis data in structured format   |
| HTML   | Visual report using Jinja2 template with tables and formatting |
| PDF    | Professional printable report created with ReportLab library   |

### Risk Scoring Algorithm:

The system calculates a risk score (0-100) based on multiple heuristics:

| Indicator                            | Points |
|--------------------------------------|--------|
| Not a PE file                        | +20    |
| YARA rule matches detected           | +30    |
| Suspicious API imports found         | +20    |
| High entropy sections ( $\geq 7.5$ ) | +15    |
| Suspicious compilation timestamp     | +15    |

### MITRE ATT&CK Mapping:

The generator maps detected malware families to MITRE ATT&CK techniques:

| Malware Family | MITRE Technique                         |
|----------------|-----------------------------------------|
| Packer         | T1027 - Obfuscated Files or Information |
| Ransomware     | T1486 - Data Encrypted for Impact       |
| Infostealer    | T1081 - Credentials in Files            |



## 6. Schemas and Models

Pydantic schemas provide strong typing, validation, and serialization throughout the application.

### 6.1 Enumerations (app/schema/enum.py)

| Enumeration           | Values                                    | Purpose                             |
|-----------------------|-------------------------------------------|-------------------------------------|
| <b>SuspicionLevel</b> | clean, suspicious                         | Classification of analysis findings |
| <b>FileReason</b>     | NOT_PE, INVALID_SIZE, CORRUPT_HEADER, OK  | Validation failure reasons          |
| <b>APICategory</b>    | memory, registry, network, process, other | Categorization of API calls         |

### 6.2 Analysis Schemas (app/schema/hashing.py)

| Schema                | Description                                                     |
|-----------------------|-----------------------------------------------------------------|
| <b>HashesResponse</b> | Contains SHA-256 hash string                                    |
| <b>SectionInfo</b>    | PE section details: name, size, entropy, suspicion level        |
| <b>ImportInfo</b>     | DLL import information: DLL name and list of imported functions |
| <b>ImportAnalysis</b> | Complete import analysis with suspicious API flagging           |
| <b>FileAnalysis</b>   | Comprehensive analysis result combining all analysis components |

#### File Analysis Fields:

- **is\_pe**: Boolean indicating valid PE format
- **valid\_size**: Boolean for size validation
- **sha256**: File hash identifier
- **compile\_time**: Timestamp from PE header
- **timestamp\_suspicious**: Anomalous timestamp flag

- sections: List of SectionInfo objects
- imports: List of ImportInfo objects
- suspicious\_imports: Flagged dangerous APIs
- has\_signature: Digital signature presence
- reason: FileReason enum value
- iocs: IOCResult object with extracted indicators
- yara\_results: YaraResult object with rule matches

### 6.3 YARA Schemas (app/schema/yara.py)

| Schema                 | Description                                                          |
|------------------------|----------------------------------------------------------------------|
| <b>YaraStringMatch</b> | Individual string match with offset, identifier, and matched content |
| <b>YaraMatch</b>       | Complete rule match with name, tags, metadata, and string matches    |
| <b>YaraResult</b>      | Collection of all matches with total count                           |

### 6.4 IOC Schemas (app/schema/IOC.py)

IOCResult schema contains lists for each IOC type:

- urls: List[str] - HTTP/HTTPS URLs
- ips: List[str] - IPv4 addresses
- domains: List[str] - Domain names
- emails: List[str] - Email addresses
- registry\_keys: List[str] - Windows registry paths
- file\_paths: List[str] - Windows file system paths

## 7. Detection Rules

### 7.1 YARA Rules Overview

YARA rules enable pattern-based detection of known malware families and behaviors. Rules are stored in `app/rules/` directory and loaded at application startup.

#### Rule Structure:

```
rule RuleName
{
    meta:
        author = "analyst name"
        description = "rule purpose"
        family = "malware family"
        severity = "low|medium|high"

    strings:
        $string1 = "pattern" nocase
        $string2 = { hex pattern }

    condition:
        uint16(0) == 0x5A4D and any of them
}
```

## 7.2 Rule Examples

### C2\_Network\_Beaconing

Detects command and control communication patterns commonly used by botnets and RATs.

```
rule C2_Network_Beaconing
{
    meta:
        author = "wail"
        description = "Detects suspicious C2 communication
patterns"
        family = "C2.Beacon"
        severity = "medium"

    strings:
        $a3 = "/gate.php" nocase
        $a4 = "/panel/login.php" nocase
        $a5 = "/upload.php" nocase
        $a6 = "bot_id=" nocase
        $a7 = "cmd=" nocase

    condition:
        uint16(0) == 0x5A4D and (any of ($a3,$a4,$a5,$a6,$a7))
}
```

### CoinMiner\_Generic

Identifies cryptocurrency mining malware based on mining pool connections and wallet indicators.

```
rule CoinMiner_Generic
{
    meta:
        author = "wail"
        description = "Detects common cryptocurrency mining code"
        family = "CoinMiner"
        severity = "medium"

    strings:
        $a1 = "stratum+tcp://" nocase
        $a2 = "xmr" nocase
```

```
$a3 = "pool.minexmr.com" nocase
$a4 = "wallet" nocase
$a5 = "xmrig" nocase
$a6 = "cpuminer" nocase

condition:
    uint16(0) == 0x5A4D and (any of them)
}
```

## 8. Key Features

**Comprehensive PE Analysis:** Complete parsing of Windows PE file format including headers, sections, imports, and resources.

**Multi-Layer Detection:** Combines hash-based, signature-based (YARA), and heuristic detection methods.

**Entropy-Based Packing Detection:** Identifies packed or encrypted sections using Shannon entropy calculations.

**IOC Extraction:** Automatic extraction of network indicators, file paths, and registry keys using regex patterns.

**Risk Scoring:** Algorithmic risk assessment based on multiple indicators with 0-100 scoring scale.

**MITRE ATT&CK Integration:** Maps detected threats to MITRE ATT&CK framework techniques for threat intelligence.

**Multi-Format Reporting:** Generates professional reports in JSON (machine-readable), HTML (web viewing), and PDF (printable).

**Distributed Storage:** S3-compatible MinIO storage for scalable file and report management.

**Asynchronous Processing:** Built on FastAPI with async/await for high-performance concurrent request handling.

**Extensible Rule System:** Easy addition of new YARA rules without code changes - just drop .yar files in rules directory.

**Database Indexing:** MongoDB with hash indexing for fast duplicate detection and historical queries.

**API-First Design:** RESTful API enables integration with other security tools and workflows.

## 9. Dependencies and Libraries

| Library                 | Version  | Purpose                                    |
|-------------------------|----------|--------------------------------------------|
| <b>Fastapi</b>          | Latest   | Asynchronous web framework and REST API    |
| <b>Uvicorn</b>          | Latest   | ASGI web server for FastAPI                |
| <b>Pydantic</b>         | Latest   | Data validation and settings management    |
| <b>Beanie</b>           | Latest   | Async MongoDB ODM (Object Document Mapper) |
| <b>Motor</b>            | Latest   | Async MongoDB driver                       |
| <b>Minio</b>            | Latest   | S3-compatible object storage client        |
| <b>Pefile</b>           | Latest   | PE file format parser and analyzer         |
| <b>yara-python</b>      | Latest   | YARA pattern matching engine               |
| <b>jinja2</b>           | Latest   | Template engine for HTML reports           |
| <b>Reportlab</b>        | Latest   | PDF document generation library            |
| <b>python-multipart</b> | Latest   | Multipart form data parser                 |
| <b>Asyncio</b>          | Built-in | Asynchronous I/O framework                 |
| <b>Hashlib</b>          | Built-in | Cryptographic hashing functions            |
| <b>Re</b>               | Built-in | Regular expression operations              |
| <b>Datetime</b>         | Built-in | Date and time handling                     |
| <b>Pathlib</b>          | Built-in | Object-oriented filesystem paths           |
| <b>Subprocess</b>       | Built-in | External process execution                 |

## 10. API Documentation

### Endpoint: POST /

Upload and analyze a file for malware detection.

### Request:

Method: POST  
Content-Type: multipart/form-data  
Parameters:  
- file: UploadFile (required) - Binary file to analyze  
- generate\_report: bool (optional, default: false) - Generate reports

### Response:

Returns a FileModel object with complete analysis results:

```
{
  "file_id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "suspicious.exe",
  "size": "524288",
  "hash": "abc123...",
  "content_type": "application/x-msdownload",
  "uploaded_at": "2026-02-18T20:30:00Z",
  "analysis": {
    "is_pe": true,
    "valid_size": true,
    "sha256": "abc123...",
    "compile_time": "2024-01-15T10:30:00",
    "timestamp_suspicious": false,
    "sections": [...],
    "imports": [...],
    "suspicious_imports": ["VirtualAlloc", "CreateRemoteThread"],
    "has_signature": false,
    "yara_results": {
      "matches": [...],
      "total_matches": 2
    },
  },
  "iocs": {
    "urls": ["http://malicious.com/gate.php"],
    "ips": ["192.168.1.100"],
    "domains": ["evil.com"],
  },
}
```



```
    ...  
  }  
}  
}
```

### Error Responses:

- 400 Bad Request: Known malware hash detected in blacklist
- 500 Internal Server Error: Analysis failure or system error

## 11. Security Considerations

**Malware Containment:** All uploaded files are treated as potentially malicious. Analysis is performed in isolated processes with no execution of uploaded binaries.

**Hash-Based Blacklisting:** Known malware hashes are blocked immediately to prevent storage and analysis of already identified threats.

**Temporary File Cleanup:** Uploaded files are stored temporarily during analysis and cleaned up after processing to prevent disk accumulation.

**Authentication Required:** Production deployments should implement authentication on the API endpoint to prevent unauthorized access.

**Rate Limiting:** Consider implementing rate limiting to prevent abuse and resource exhaustion attacks.

**Input Validation:** File size limits (1KB - 50MB) prevent resource exhaustion from extremely large files.

**Database Security:** MongoDB should be configured with authentication and encryption in production environments.

**Object Storage Security:** MinIO buckets should use access policies and encryption for sensitive malware samples.

**Report Access Control:** Presigned URLs for reports have time-limited access to prevent unauthorized data exposure.

**Logging and Auditing:** All analysis operations should be logged for security auditing and forensic purposes.

## 12. Conclusion

Static Malware Analyzer provides a robust, scalable platform for static analysis of Windows PE files. By combining multiple detection techniques—hash-based blacklisting, structural analysis, YARA pattern matching, and IOC extraction—the system offers comprehensive threat detection capabilities.

The modular architecture enables easy extension with new YARA rules, additional analysis techniques, and integration with other security tools. The asynchronous design ensures high performance under load, while the distributed storage architecture provides scalability for large-scale malware analysis operations.

## Contact and Resources

GitHub Repository: <https://github.com/Githu6Girl/StaticMalwareAnalyzer>

Members:

- Bentafat Wail
- Benbraham Sara
- Saiki Meriem
- Louail Wissam

Documentation: See README.md in the repository